The University of New Mexico

recap

- **Homogeneous Coordinates Representation（齐次坐标表示）**
  - 点 Point(x,y,z,w) : w!=0,
  - 矢量 Vector(x,y,z,0)

- **Affine Transformation(仿射变换~五种直观的标准变换）**
  - Standard Transformation：Translation, Rotation, Scaling, Reflection, Shear.
  - Use Homogeneous Coordinates, Transformation is a 4*4 Matrix, 12 freedom
  - Use column vector representation , affine transformation then like:

$$M_{4*4} = \begin{bmatrix} sx & b & c & tx \\ d & sy & f & ty \\ g & h & sz & tz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Concatenation Transformation（串联变换）**

  p' = Mn*…*M2*M1* p; //Mi is Standard Transformation

  M=Mn*…*M2*M1; //M is Concatenation Transformation

  ◆ 任何复杂变换都可以表示成五种标准变换的顺序累积乘形式！

  ◆ 通常用列向量表示图形，串联变换中的标准变换是按左乘顺序进行矩阵乘的！



Today Agenda

- 1.观察流水线中包含哪些坐标系和变换
  - 五种坐标和四类变换
- 2.模型变换的推导和应用举例
- 3.观察变换的推导和应用举例



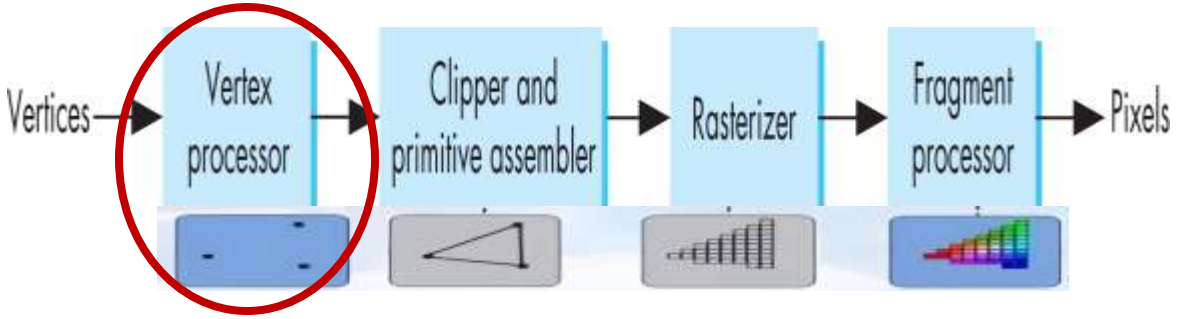
Outline

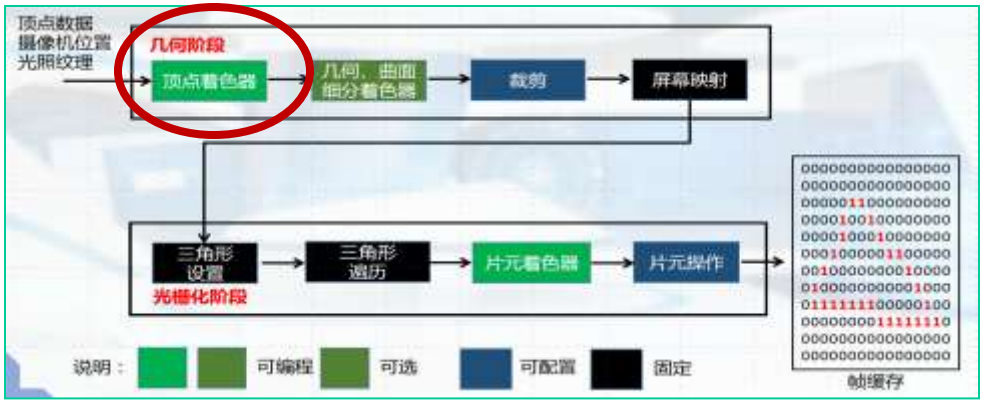
- **Viewing Process 观察流程中的串联变换**
  - **Model-View Transformation（模视变换）**
    - **Model Transformation模型变换***
    - **View Transformation观察变换***
  - Projection Transformation投影变换
  - Viewport Transformation屏幕映射

The University of New Mexico

Computer Viewing Process

➢目前学习位置:





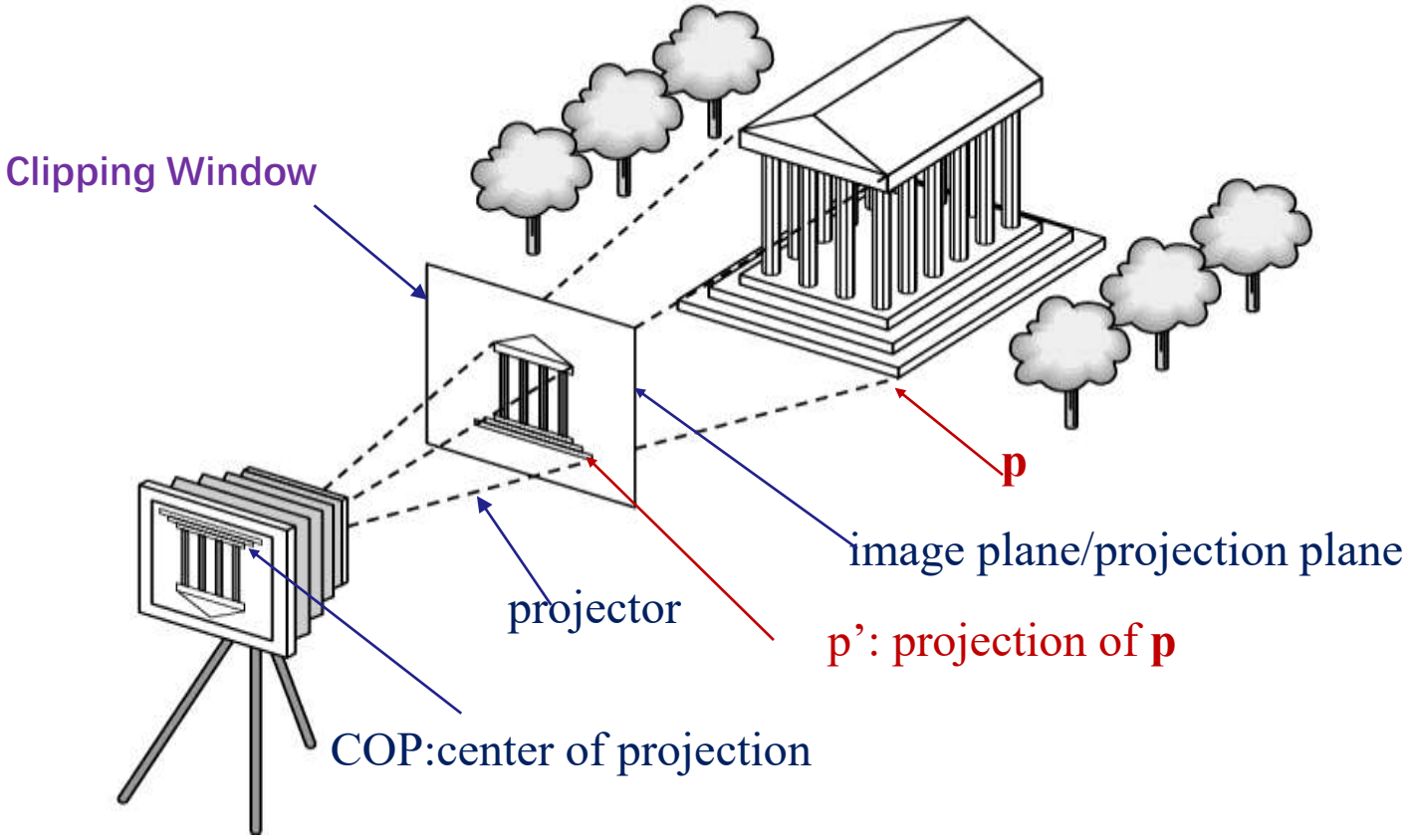


Computer Viewing Process

The University of New Mexico

- **Synthetic Camera Model虚拟相机模型**
  - **从P到P',顶点坐标如何变换？**

计算机图形学中的坐标系转换过程就像一场“3D物体的旅行”，它从自己的本地空间local出发，最终被绘制到设备屏幕的2D像素空间



Computer Viewing Process（cont.）

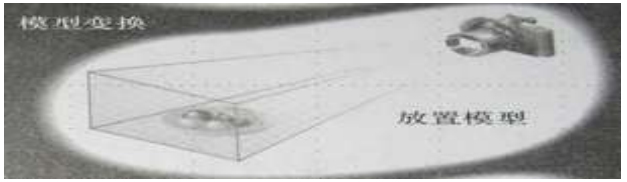
The University of New Mexico

➢类比于相机拍摄过程，经历“五”种坐标系和“四”种变换！

物体模型

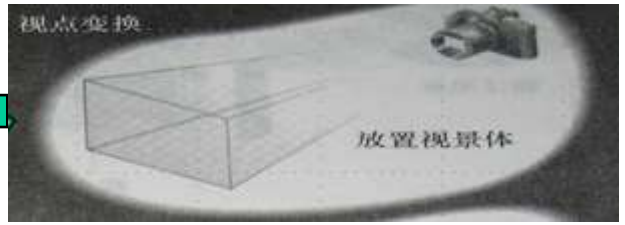
■MC模型坐标

将物体建模坐标描述转换到世界坐标场景下描述-模型变换



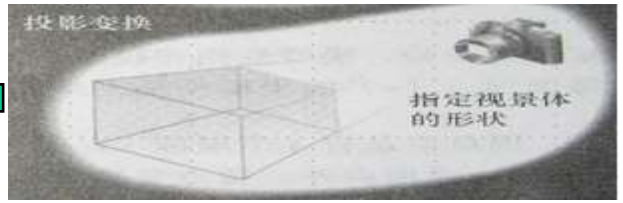
■WC世界坐标

将世界坐标场景描述转换为观察坐标系描述-观察变换/相机变换/眼坐标变换



■VC观察坐标

将观察坐标裁剪窗口内景物进行规范化投影变换-规范化投影变换+裁剪



■CC/NDC规范化坐标

将规范化区域景物映射到设备坐标系下视区内-屏幕映射/视口变换



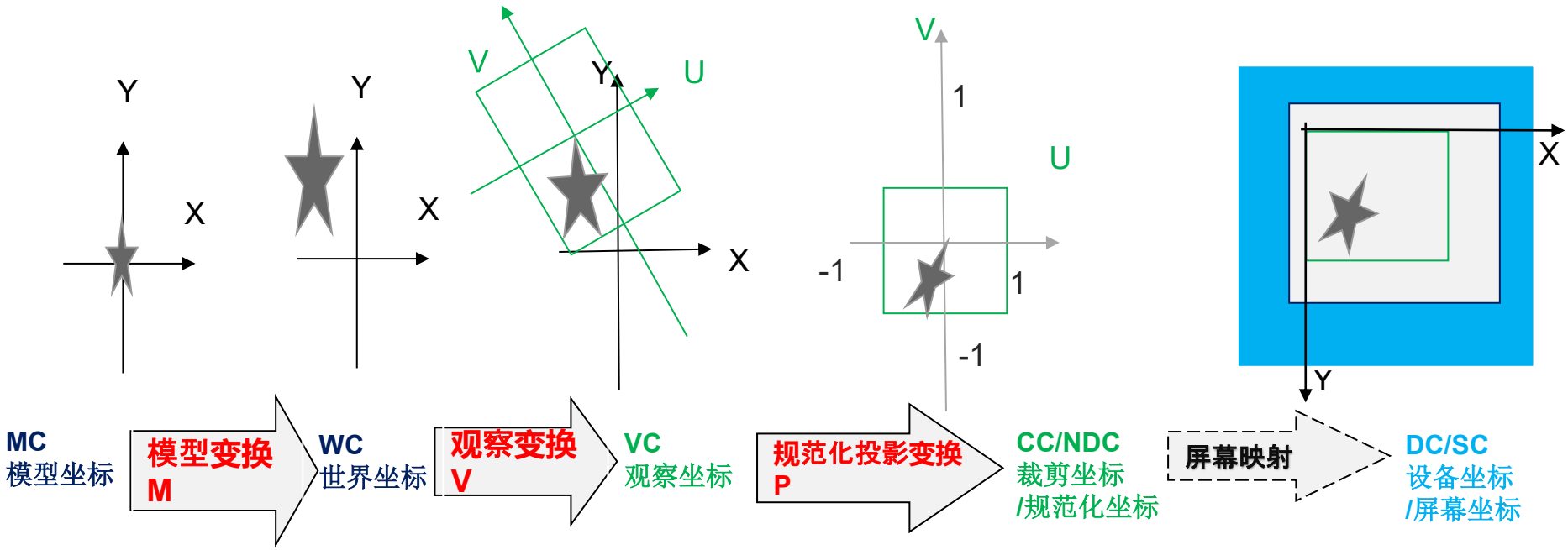
■DC设备坐标

■输出设备

Computer Viewing Process（cont.）

The University of New Mexico

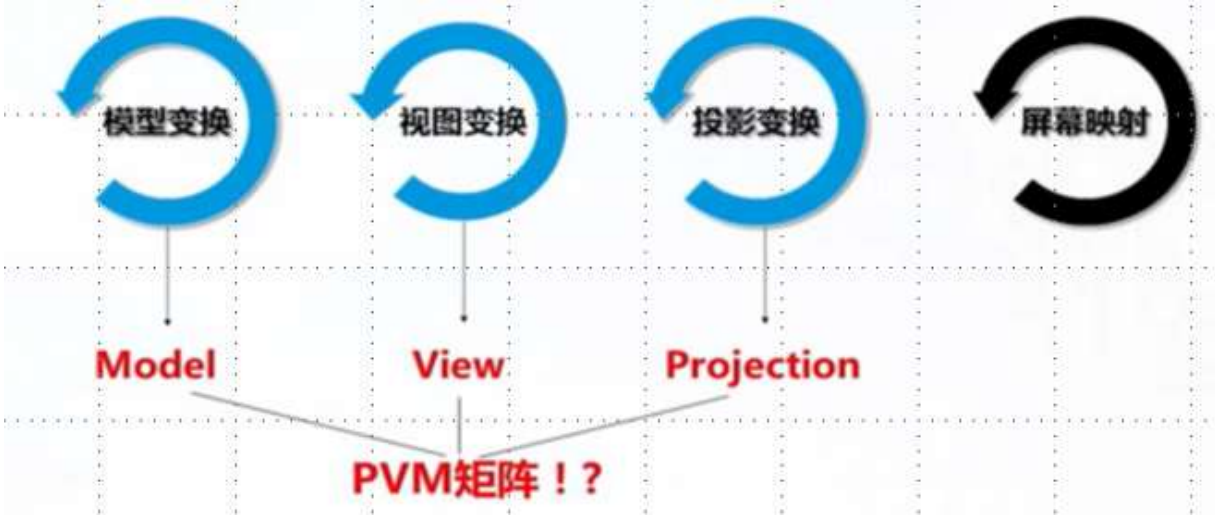
- 前三个变换： M, V, P 需要程序员编制完成，输出裁剪坐标
- 最后的屏幕映射由系统自动完成，输出屏幕坐标





Computer Viewing Process（cont.)

- MVP 或 PVM：
  - PVM = Projection Matrix * View Matrix* Model Matrix
    - 注1： 以列向量方式表示顶点和变换，所以是左乘顺序
    - 注2： Mview * Mmodel 常常合并称为 ModelView矩阵
    - 注3： 该操作通常在Vertex Shader 顶点着色器中编程完成



- Every change of coordinates is equivalent to a concentrate matrix transformation (每一个变换都是一个串联变换矩阵)



Outline

- Viewing Process 观察流程中的串联变换

- **ModelView Transformation**
  - **Model Transformation模型变换**
  - **View Transformation观察变换**

- Projection Transformation

- Viewport Transformation



ModelView Transformation

- “模视变换”由“模型变换 ”和“观察变换”构成
  - **模型变换Model Transformation：摆放物体**
  - **观察变换View Transformation：摆放相机，**
- 模型变换和观察变换具有对偶性
  - 同一画面，既可以是模型变换后结果，也可以是相机变换后的结果！

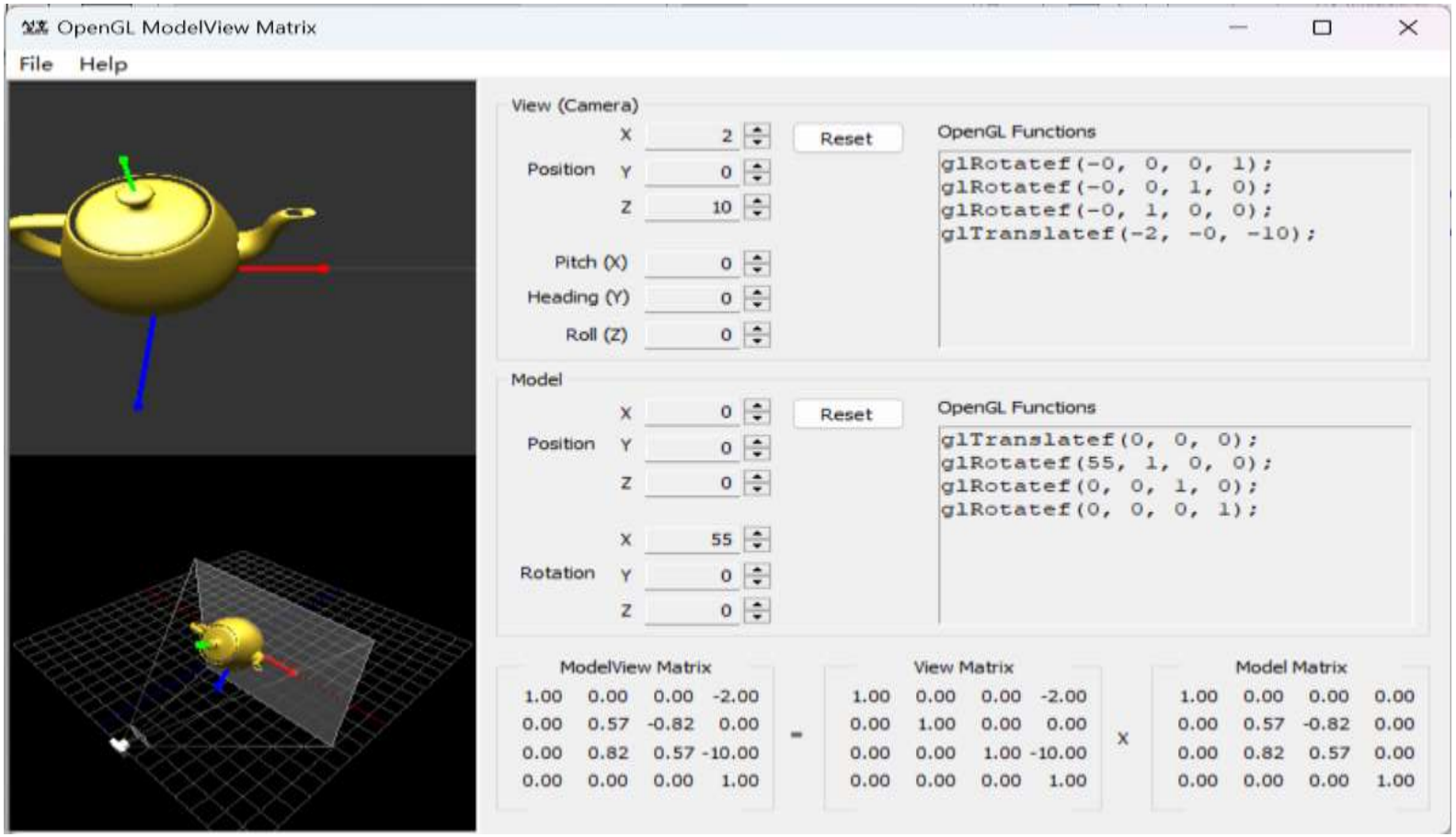






ModelView Transformation

- 模视变换矩阵
  - ModeView Matrix = View Matrix * Model Matrix





Outline

- Viewing Process 观察流程的串联变换

- **Model-View Transformation**
  - **Model Transformation模型变换***
  - **View Transformation视点变换***

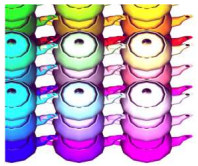
- Projection Transformation
- Viewport Transformation

The University of New Mexico

Model Transformation

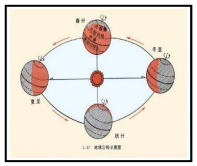
- Model Transformation模型变换
  - 将模型局部坐标转换为场景全局世界坐标(from MC to WC)

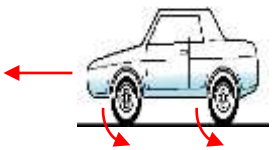
模型在建立之初，有它自身的坐标系以及相应的点坐标。如果把它放入场景中的某个位置，需要将它自身的坐标点全部移动到改位置（世界坐标系WC），这个转换过程就叫“模型变换”





- **简单的模型变换：实例变换**
  - 定义同一个模型在世界空间中的多个副本的摆放。它关心的是“这个物体的复制品，在世界里的不同位置放多少个，分别放在哪里”。







- **复杂的模型变换：层次建模**
  - **定义单个模型在其自身“模型空间”中的摆放（位置、旋转、缩放）。它关心的是“这个物体自己长什么样，摆成什么姿势”。**

Model Transformation（cont.）

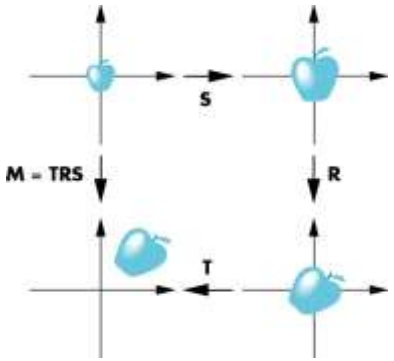
➢ Instance Transformation实例变换

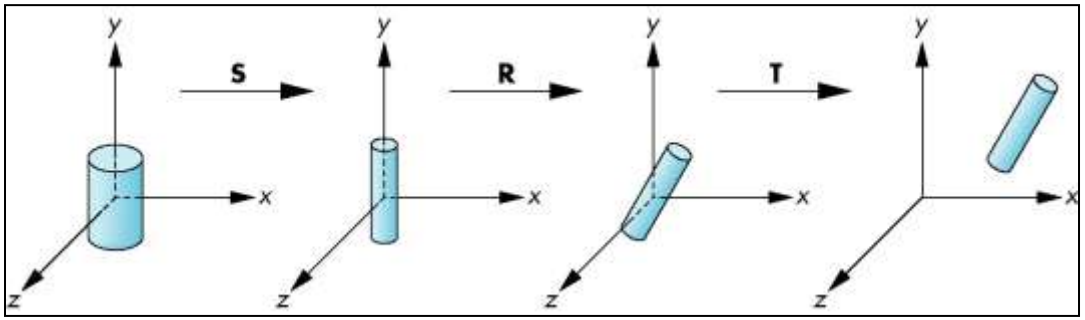
- a prototype object ~a symbol~图符(原型)
- Each appearance of the object in the model is~ an instance~实例
  - Must scale, orient, position 缩放，定向和定位
- Defines instance transformation ， **General: M=TRS**

  Scale: S

  Orient:R

  Locate:T







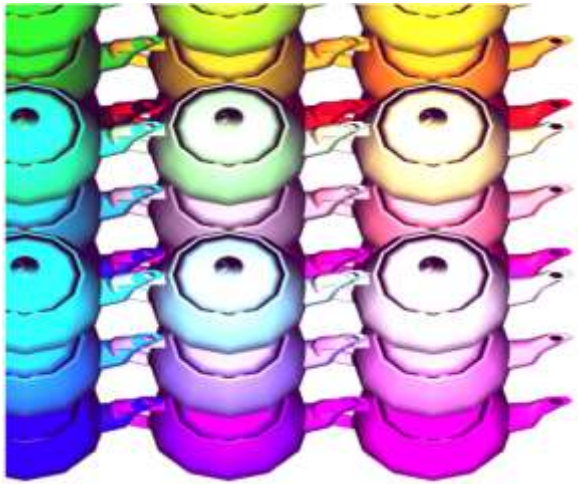
Model Transformation(cont.)

- Instance Transformation实例变换(cont.)
  - 通常为：T*R*S

Example1：WebGL2 性能优化 - 实例化绘制
https://webgl2fundamentals.org/webgl/lessons/zh_cn/webgl-instanced-drawing.html



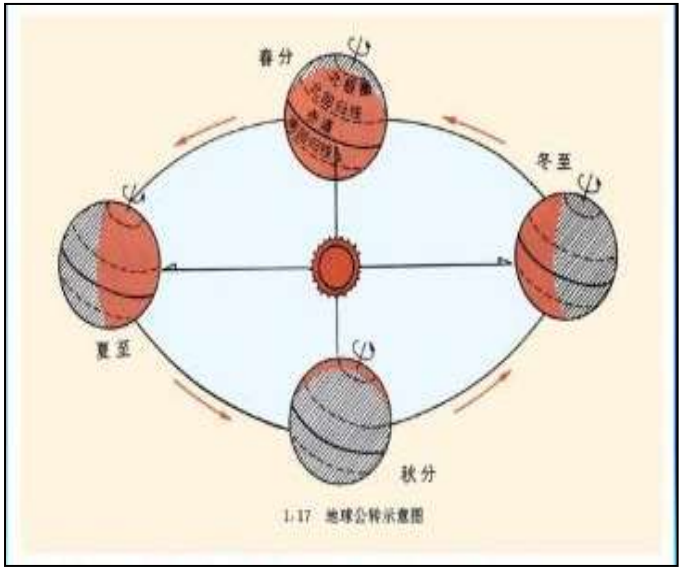
Example2:AngelCode8E/09/teapotInstance2.html

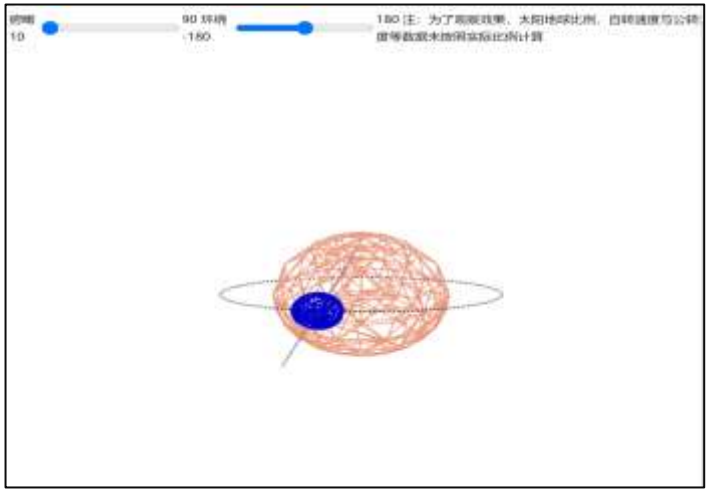


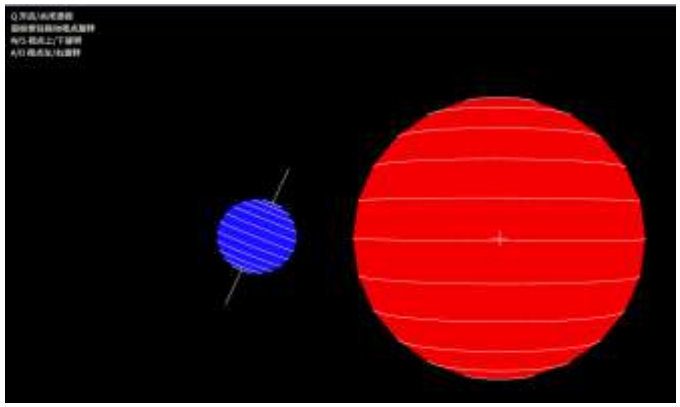
The University of New Mexico

Model Transformation(cont.)

- Complex Model Transformation 复杂的模型变换
  - ex1：Sun-planet Example（演示）



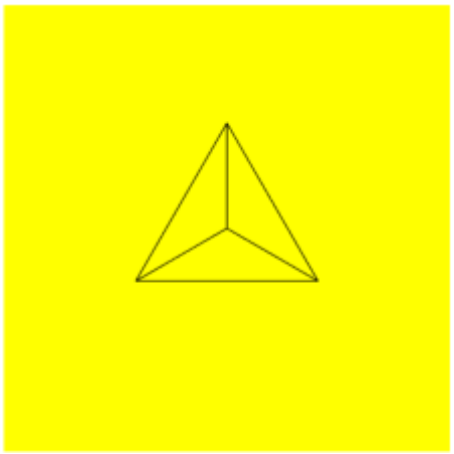
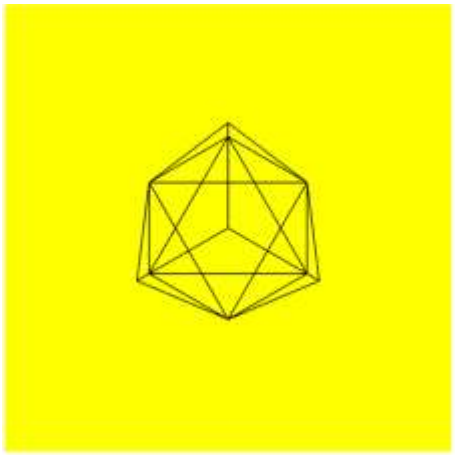
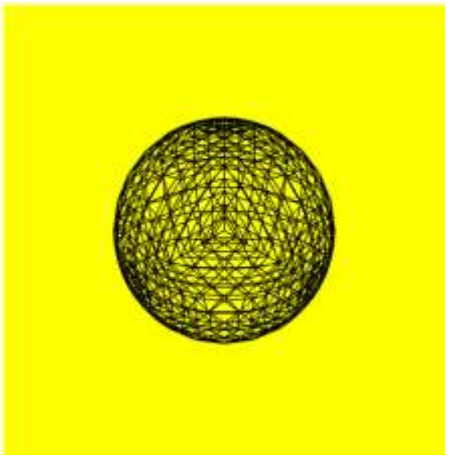




Model Transformation(cont.)

- Complex Model Transformation(cont.)
  - ex1: Sun-planet Example（cont.）
    - 球体建模：四面体递归细分得到球面网格mesh
      //ref AngleCode/chap5/ Wiresphere.js

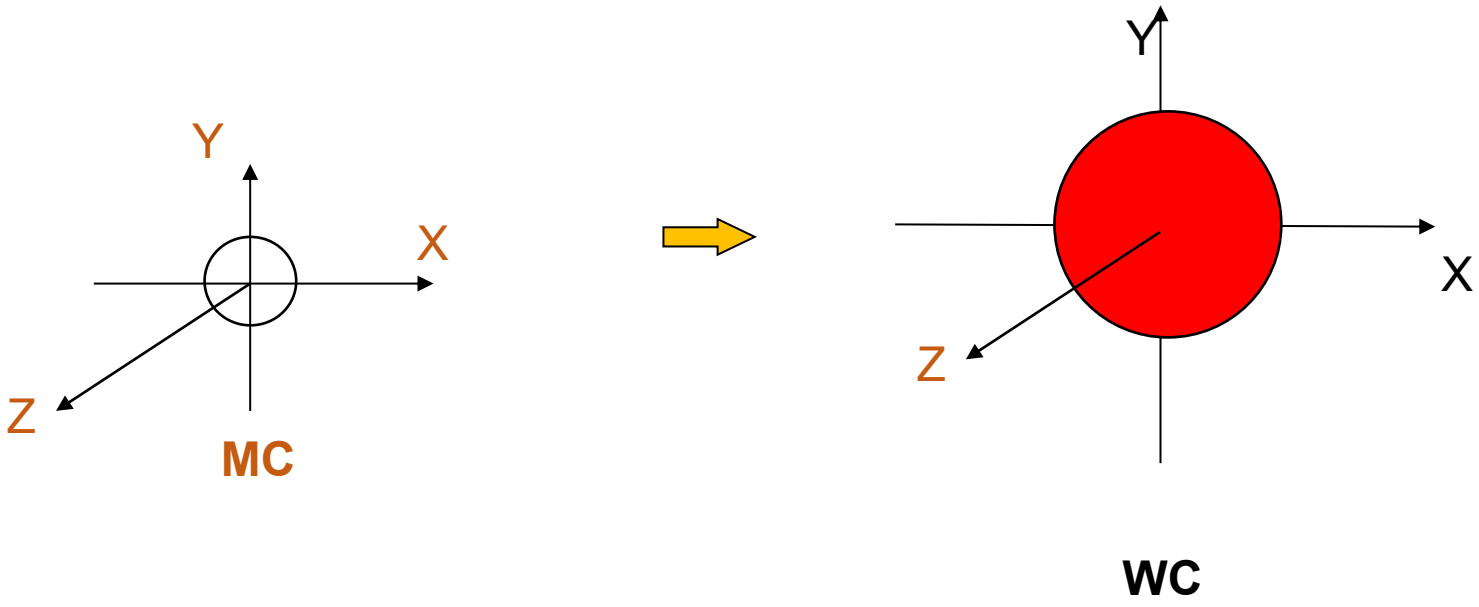
step1.初始为在单位四面体，有四个面，四个顶点在单位球面上。
Step2.取每条边上的中点P，将每个面划分为4个面，然后对新顶点进行归一化（长度归约到1，即把新顶点撑到单位球面上）
step3.重复step2得到球面网格



Model Transformation(cont.)

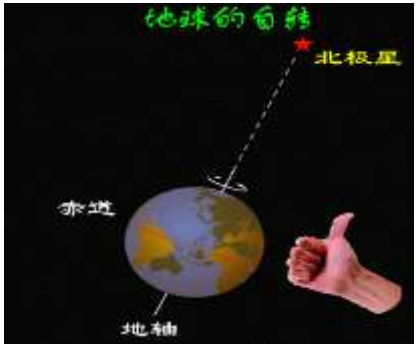
- Complex Model Transformation(cont.)
  - ex1:Sun-planet Example（cont.）
    - Msun= Scale(sx,sy,sz)

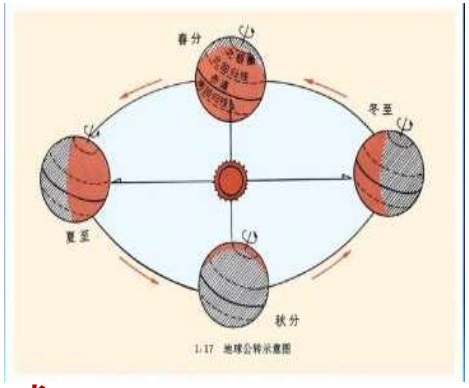


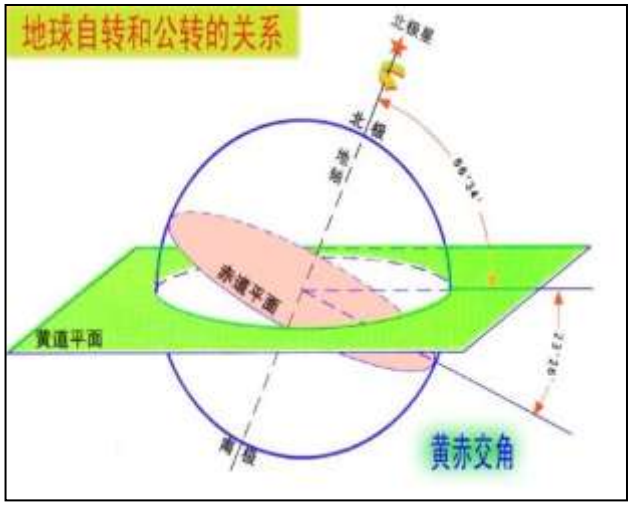
Model Transformation(cont.)

The University of New Mexico

- Complex Model Transformation(cont.)
  - ex1:Sun-planet Example（cont.）
    - **Planet:** Mplanet= ?







- 串联变换矩阵的构成：
1. 球体原型进行缩放到需要的地球尺寸：S（ sx,sy,sz ）
2. 地球绕“地轴”（y轴）逆时针“**自转**”： Ry（**alpha** ）
3. 地球**地轴的偏移**：即绕z轴顺时针旋转Rz(-phi) //(*Phi = 23.26*)
4. 地球绕太阳的公转的变换矩阵= ？

Earth'=（ ？ ）* **Rz(-Phi) Ry(alpha)** S(sx,sy,sz) **Earth**

The University of New Mexico

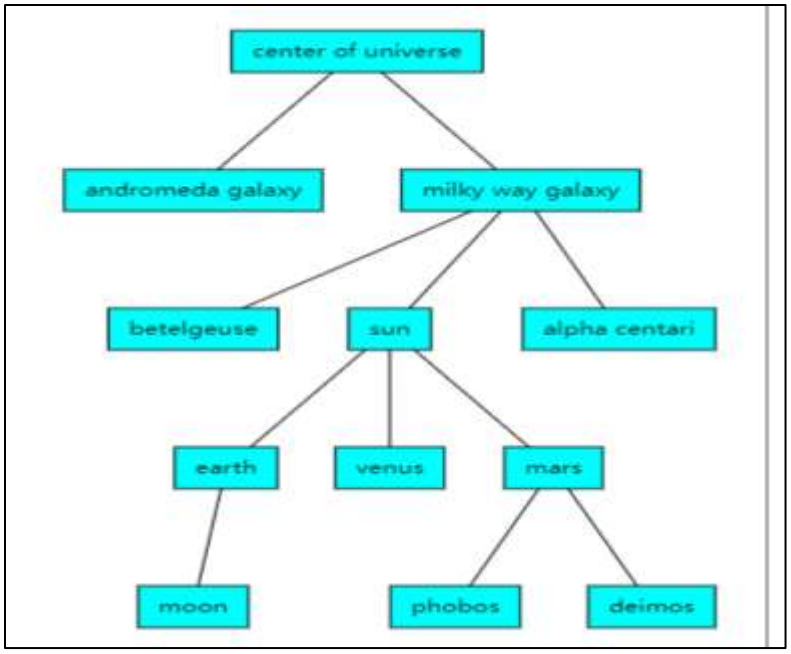
Model Transformation（cont.）

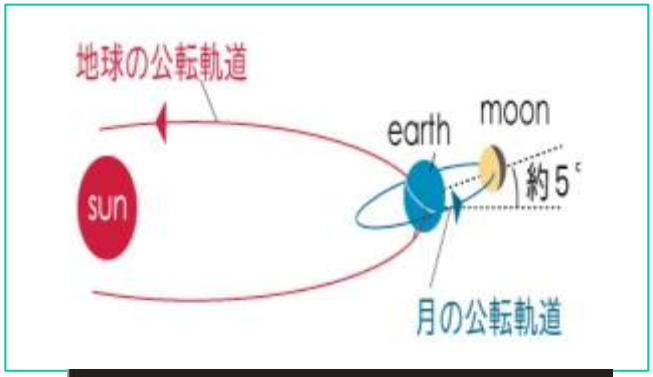
- Complex Model Transformation(cont.)
  - ex1:Sun-planet Example（cont.）

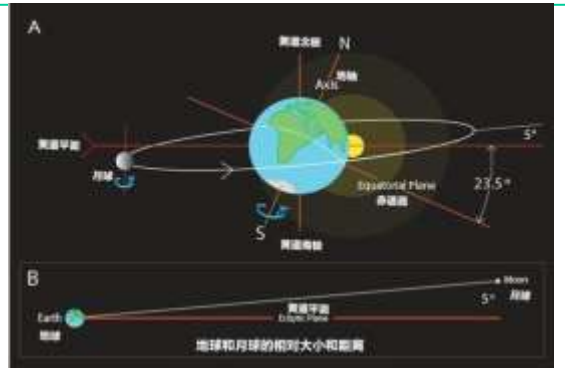
扩展：太阳-地球-月亮.

问题：月亮和地球一起绕着太阳转如何表达？ -层次结构-

https://webgl2fundamentals.org/webgl/lessons/zh_cn/webgl-scene-graph.html

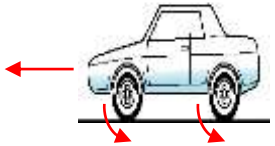






Model Transformation（cont.）

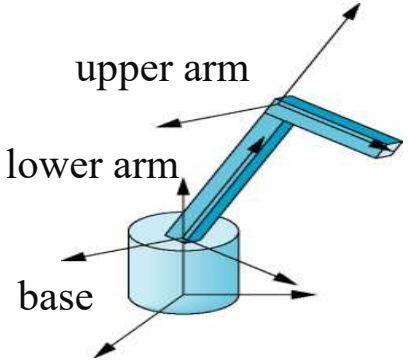
➢Complex Model Transformation(cont.)



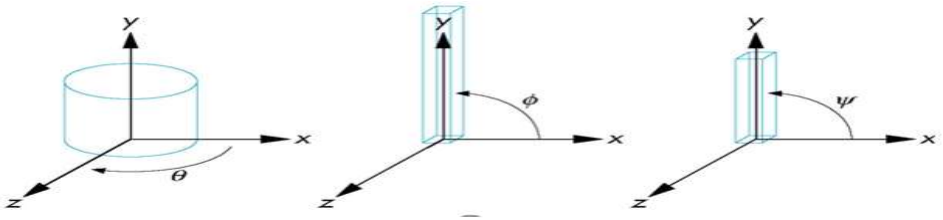
➢用模型变换实现“层次建模”(铰链模型)

◆Ex2: Relationships in Robot Arm and Required Matrices

- ➢ Rotation of base: $\mathbf{R}_b$
  - ➢ Apply $\mathbf{M} = \mathbf{R}_b$ to base
- ➢ Translate lower arm relative to base: $\mathbf{T}_{lu}$
- ➢ Rotate lower arm around joint: $\mathbf{R}_{lu}$
  - ■ Apply $\mathbf{M} = \mathbf{R}_b \mathbf{T}_{lu} \mathbf{R}_{lu}$ to lower arm
- ➢ Translate upper arm relative to upper arm: $\mathbf{T}_{uu}$
- ➢ Rotate upper arm around joint: $\mathbf{R}_{uu}$
  - ■ Apply $\mathbf{M} = \mathbf{R}_b \mathbf{T}_{lu} \mathbf{R}_{lu} \mathbf{T}_{uu} \mathbf{R}_{uu}$ to upper arm



robot arm



Base lower arm upper arm

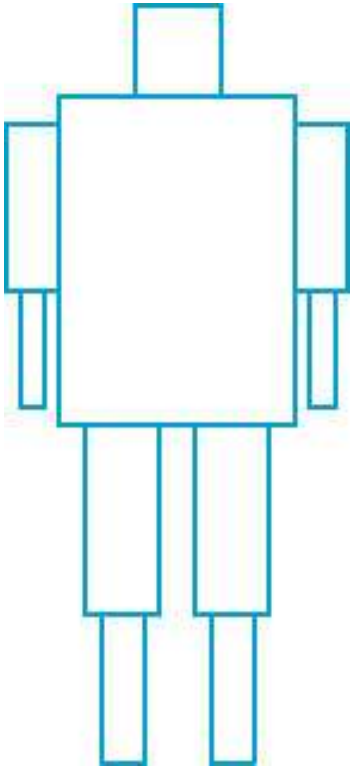
The University of New Mexico

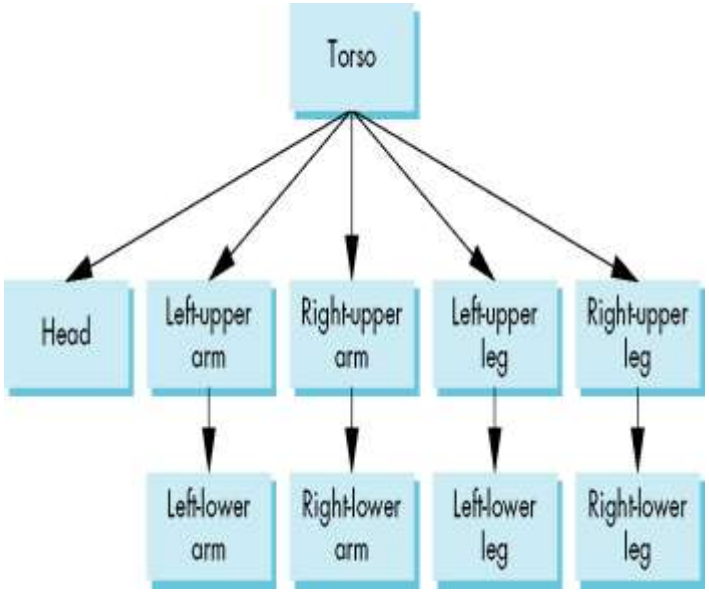
Model Transformation（cont.）

➢Complex Model Transformation(cont.)

➢Build a tree-structured model of a humanoid figure(类人机器人)

- 机器人模拟（具有铰链结构）Ref：Angel8ECode/09/figure.html(省略)

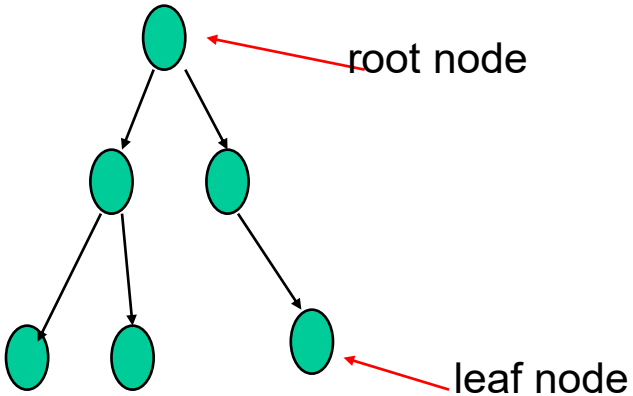


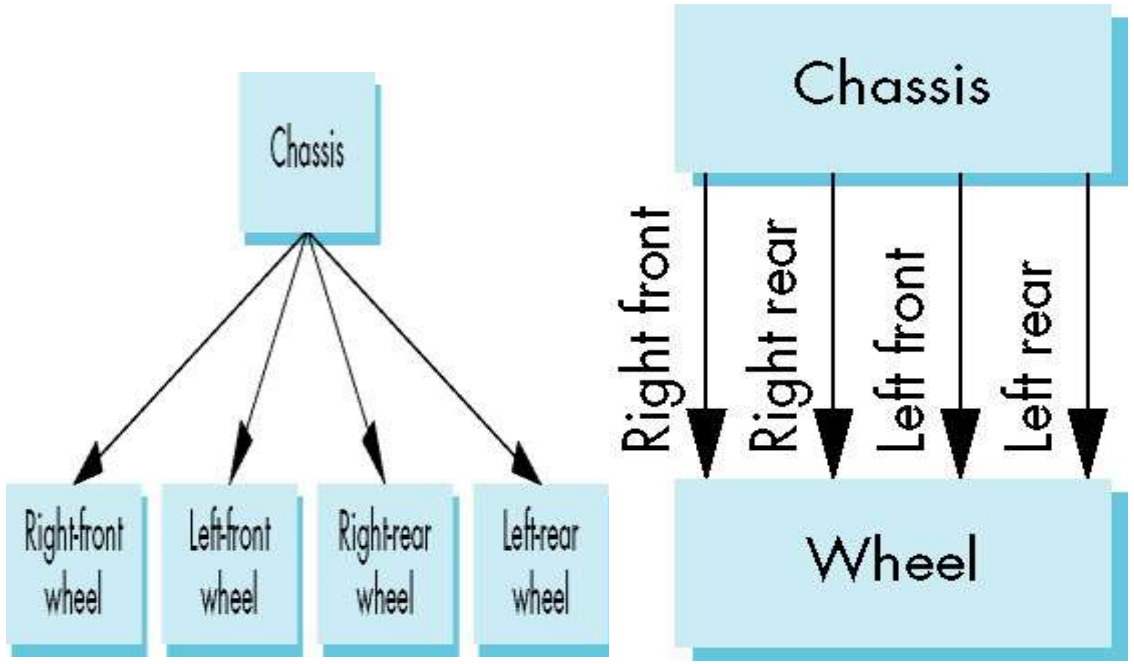




Model Transformation（cont.）

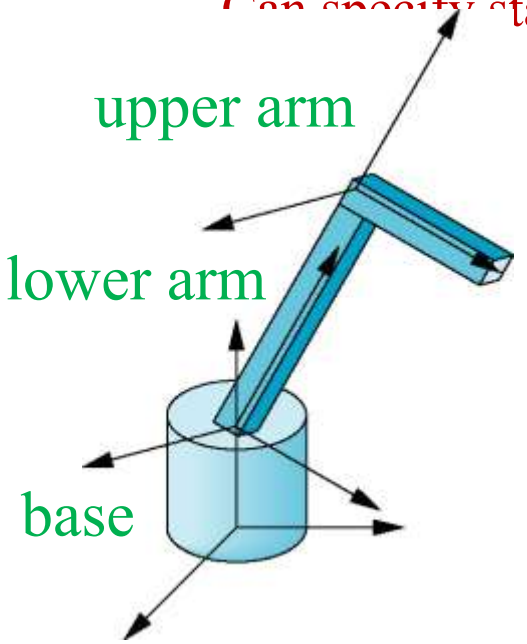
- Hierarchical Models层级模型
  - Tree: Graph in which
    - each node has exactly one parent node(except the root)
    - each node may have multiple children(Leaf or terminal node: no children)
  - Tree Model of Car

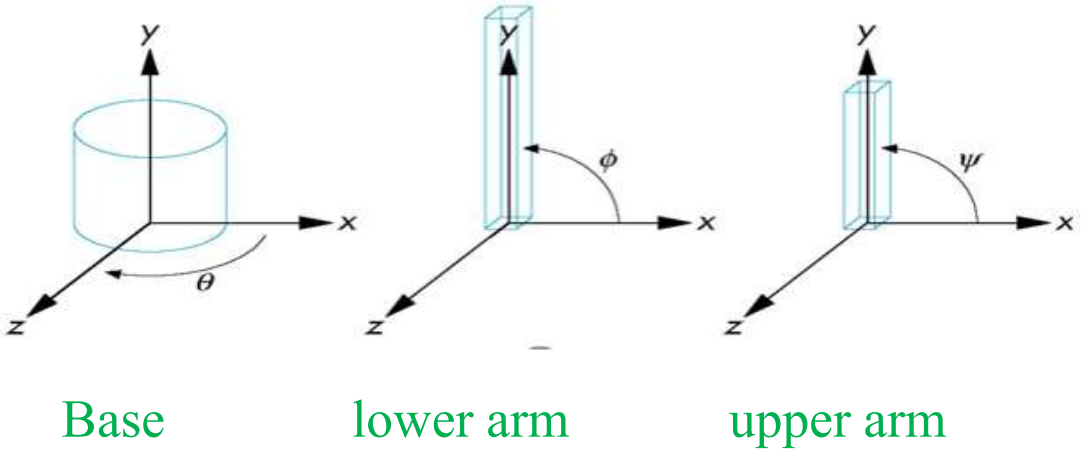




Model Transformation（cont.)

- Hierarchical models层级模型(cont.)
- Ex: Robot arm is an example of an *articulated model*(铰接模型）
  - Parts connected at joints
  - Can specify state of model by giving all joint angles





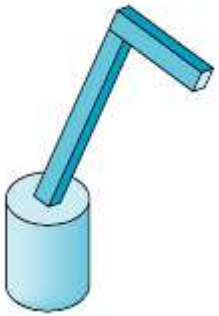
robot arm

parts in their own coodinate systems

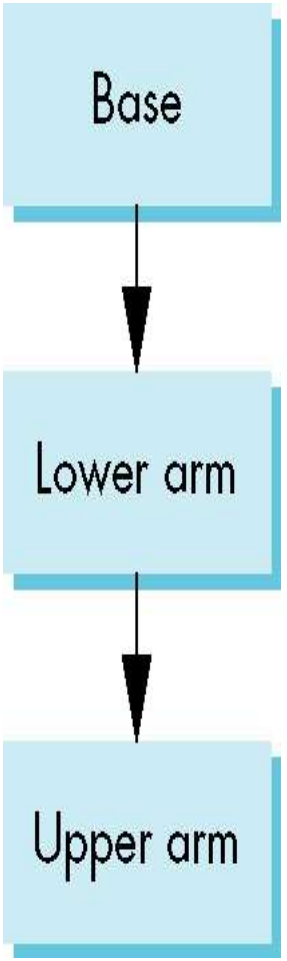


Model Transformation（cont.）

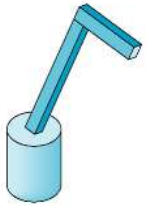
- Hierarchical models层级模型(cont.)
- Modeling with Trees
  - Nodes结点:
    - What to draw,
    - Pointers to children
  - Edges边
    - May have information on incremental changes to transformation matrices (can also store in nodes)



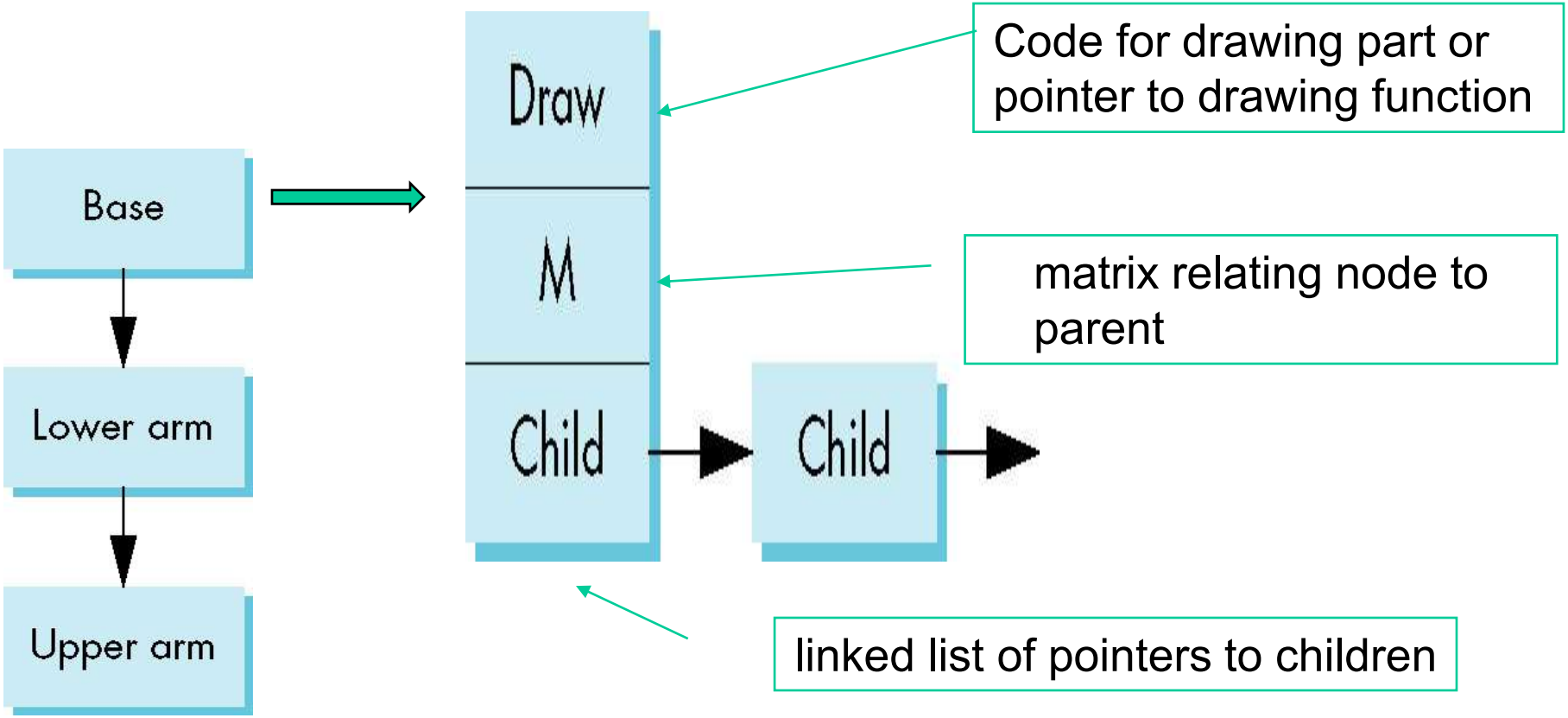
robotArm



Possible Node Structure

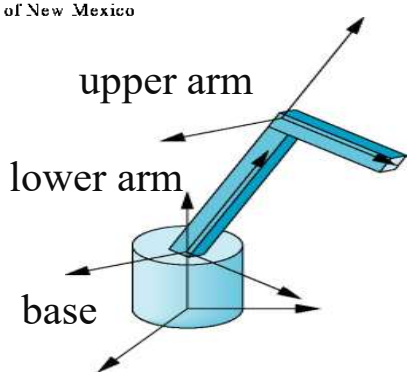


- 把所有信息都存储在节点node中，则每个节点至少包含下面3项目信息：

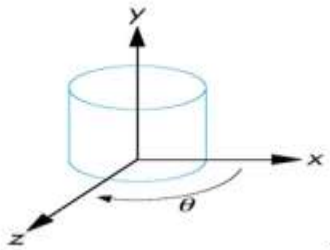


The University of New Mexico

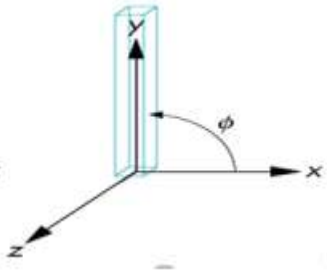
Relationships in Robot Arm and Required Matrices



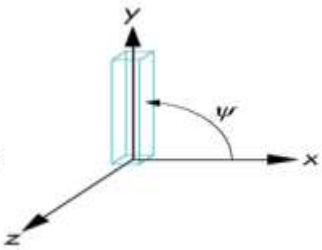
robot arm



Base



lower arm



upper arm

- Rotation of base: $\mathbf{R}_b$
  - Apply $\mathbf{M} = \mathbf{R}_b$ to base
- Translate lower arm relative to base: $\mathbf{T}_{lu}$
- Rotate lower arm around joint: $\mathbf{R}_{lu}$
  - Apply $\mathbf{M} = \mathbf{R}_b \mathbf{T}_{lu} \mathbf{R}_{lu}$ to lower arm
- Translate upper arm relative to upper arm: $\mathbf{T}_{uu}$
- Rotate upper arm around joint: $\mathbf{R}_{uu}$
  - Apply $\mathbf{M} = \mathbf{R}_b \mathbf{T}_{lu} \mathbf{R}_{lu} \mathbf{T}_{uu} \mathbf{R}_{uu}$ to upper arm

Building the Model



- robotArm: 三个实例instance，每个都采用六面体图符symbol

```
/*****平行六面体的八个顶点的位置**********************/
var vertices = [
    vec4( -0.5, -0.5,  0.5, 1.0 ),
    vec4( -0.5,  0.5,  0.5, 1.0 ),
    vec4(  0.5,  0.5,  0.5, 1.0 ),
    vec4(  0.5, -0.5,  0.5, 1.0 ),
    vec4( -0.5, -0.5, -0.5, 1.0 ),
    vec4( -0.5,  0.5, -0.5, 1.0 ),
    vec4(  0.5,  0.5, -0.5, 1.0 ),
    vec4(  0.5, -0.5, -0.5, 1.0 )
];

/*****平行六面体的八个顶点的颜色**********************/
// RGBA colors
var vertexColors = [
    vec4( 0.0, 0.0, 0.0, 1.0 ),  // black
    vec4( 1.0, 0.0, 0.0, 1.0 ),  // red
    vec4( 1.0, 1.0, 0.0, 1.0 ),  // yellow
    vec4( 0.0, 1.0, 0.0, 1.0 ),  // green
    vec4( 0.0, 0.0, 1.0, 1.0 ),  // blue
    vec4( 1.0, 0.0, 1.0, 1.0 ),  // magenta
    vec4( 1.0, 1.0, 1.0, 1.0 ),  // white
    vec4( 0.0, 1.0, 1.0, 1.0 )   // cyan
];
```



Instance=modelViewMatrix*instanceMatrix*Symbol)

- instanceMatrix某实例变换矩阵；
- modelviewMatrix累积的当前”模视变换矩阵”



```
// Parameters controlling the size of the Robot's arm
var BASE_HEIGHT      = 2.0;
var BASE_WIDTH       = 5.0;

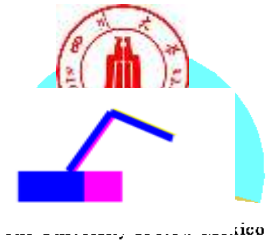
var LOWER_ARM_HEIGHT = 5.0;
var LOWER_ARM_WIDTH  = 0.5;

var UPPER_ARM_HEIGHT = 5.0;
var UPPER_ARM_WIDTH  = 0.5;
```

```
//----------------------------------------------------------------------------
function base() {
    var s = scale4(BASE_WIDTH, BASE_HEIGHT, BASE_WIDTH);
    var instanceMatrix = mult( translate( 0.0, 0.5 * BASE_HEIGHT, 0.0 ), s);
    var t = mult(modelViewMatrix, instanceMatrix);
    gl.uniformMatrix4fv(modelViewMatrixLoc,  false, flatten(t) );
    gl.drawArrays( gl.TRIANGLES, 0, NumVertices );
}

//----------------------------------------------------------------------------
function upperArm() {
    var s = scale4(UPPER_ARM_WIDTH, UPPER_ARM_HEIGHT, UPPER_ARM_WIDTH);
    var instanceMatrix = mult(translate( 0.0, 0.5 * UPPER_ARM_HEIGHT, 0.0 ),s);
    var t = mult(modelViewMatrix, instanceMatrix);
    gl.uniformMatrix4fv( modelViewMatrixLoc,  false, flatten(t) );
    gl.drawArrays( gl.TRIANGLES, 0, NumVertices );
}

//----------------------------------------------------------------------------
function lowerArm()
{
    var s = scale4(LOWER_ARM_WIDTH, LOWER_ARM_HEIGHT, LOWER_ARM_WIDTH);
    var instanceMatrix = mult( translate( 0.0, 0.5 * LOWER_ARM_HEIGHT, 0.0 ), s);
    var t = mult(modelViewMatrix, instanceMatrix);
    gl.uniformMatrix4fv( modelViewMatrixLoc,  false, flatten(t) );
    gl.drawArrays( gl.TRIANGLES, 0, NumVertices );
}
```



Rendering Robot Arm

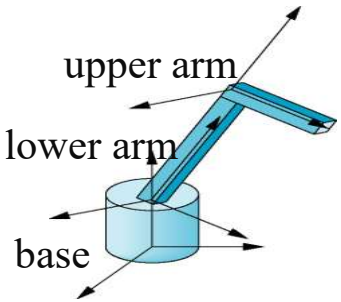
```
var render = function() {
    gl.clear( gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT );

    modelViewMatrix = rotate(theta[Base], 0, 1, 0 );
    base();

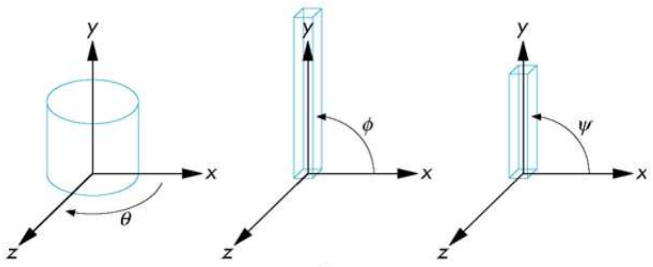
    modelViewMatrix = mult(modelViewMatrix, translate(0.0, BASE_HEIGHT, 0.0));
    modelViewMatrix = mult(modelViewMatrix, rotate(theta[LowerArm], 0, 0, 1 ));
    lowerArm();

   modelViewMatrix  = mult(modelViewMatrix, translate(0.0, LOWER_ARM_HEIGHT,0.0));
   modelViewMatrix  = mult(modelViewMatrix,rotate(theta[UpperArm], 0, 0, 1) );
   upperArm();

   requestAnimFrame(render);
}
```



robot arm



Base

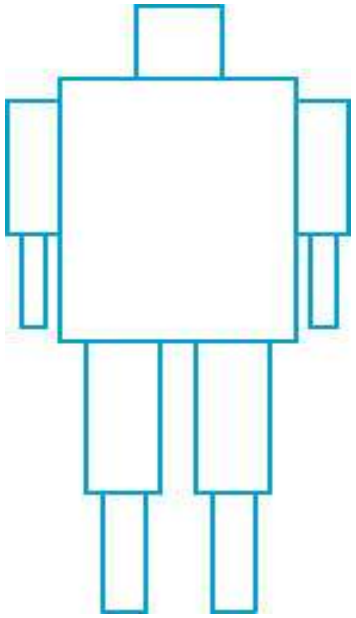
lower arm

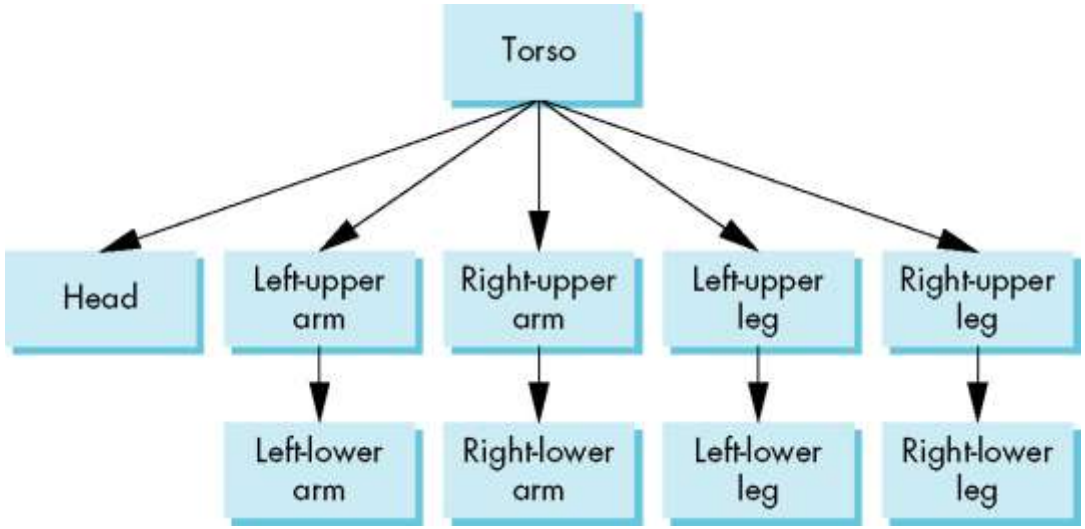
upper arm

The University of New Mexico

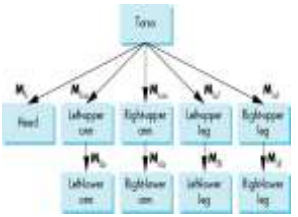
Model Transformation（cont.）

- Build a tree-structured model of a humanoid figure(类人机器人)
- //See
  - Examine various traversal strategies 探讨各种遍历策略
  - Build a generalized tree-model structure that is independent of the particular model创建一个独立于特定模型的更通用的树结构模型





Model Transformation（cont.）



Stack-based Traversal Code ,类似于前面的arm，arm没有兄弟结点，这里有不少兄弟，所以用了栈结构。但是当图形变化时需要重新写过程，不通用。

```
figure() {
    PushMatrix()
    torso();
    Rotate (…);
    head();
    PopMatrix();
    PushMatrix();
    Translate(…);
    Rotate(…);
    left_upper_arm();
    PopMatrix();
    PushMatrix();
```

save present model-view matrix

update model-view matrix for head

recover original model-view matrix

save it again

update model-view matrix for left upper arm

recover and save original model-view matrix again

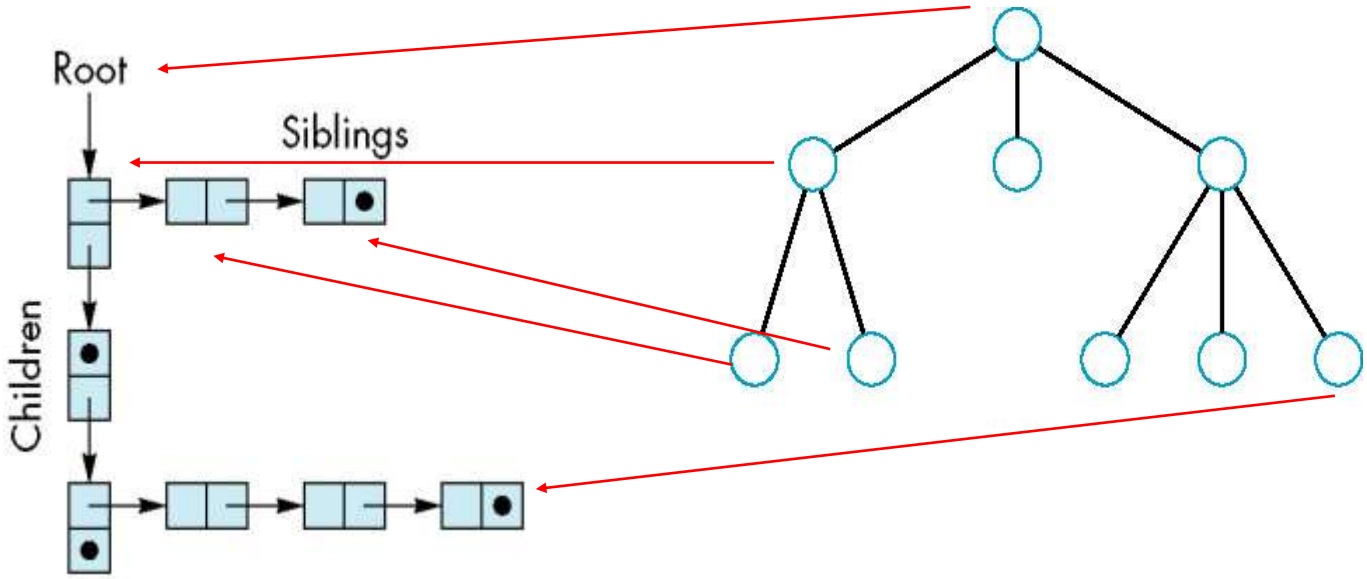
rest of code



Model Transformation（cont.）

- Generalizations（泛化）
  - Need to deal with multiple children
    - How do we represent a more general tree? How do we traverse such a data structure?
  - Animation
    - How to use dynamically? Can we create and delete nodes during execution?
- Need a data structure to represent tree and an algorithm to traverse the tree 需要使用更强大的树层级结构实现方法！

***left-child right sibling* structure（左孩子右兄弟链表结构）**





Model Transformation（cont.）

The University of New Mexico

```
function traverse(Id) {
   if(Id == null) return;
   stack.push(modelViewMatrix);
   modelViewMatrix = mult(modelViewMatrix, figure[Id].transform);
   figure[Id].render();
  //子节点递归绘制，需要弹栈
   if(figure[Id].child != null) traverse(figure[Id].child);
   modelViewMatrix = stack.pop();
 //兄弟节点绘制
   if(figure[Id].sibling != null) traverse(figure[Id].sibling);
}
```

```
var render = function() {
        gl.clear( gl.COLOR_BUFFER_BIT );
        traverse(torsoId);
        requestAnimFrame(render);
}
```



Outline

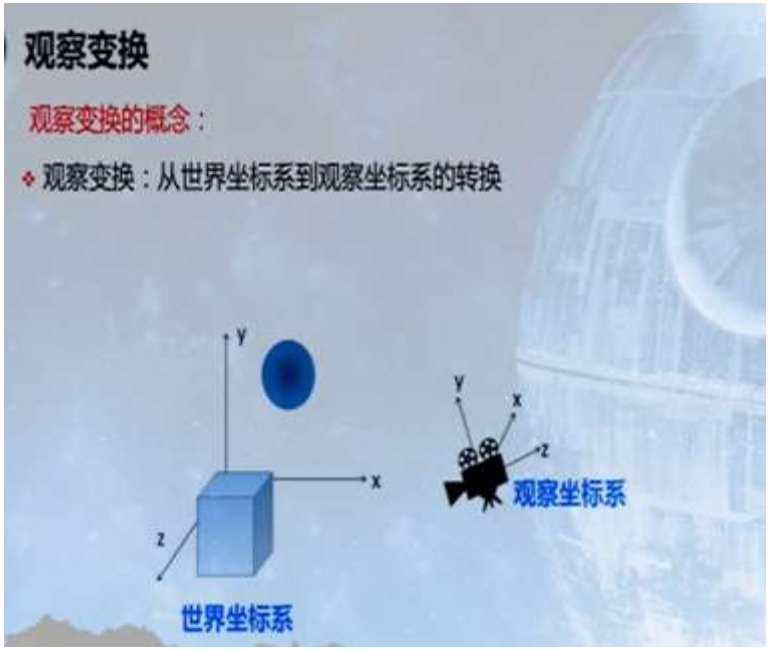
-Viewing Process观察流程的串联变换

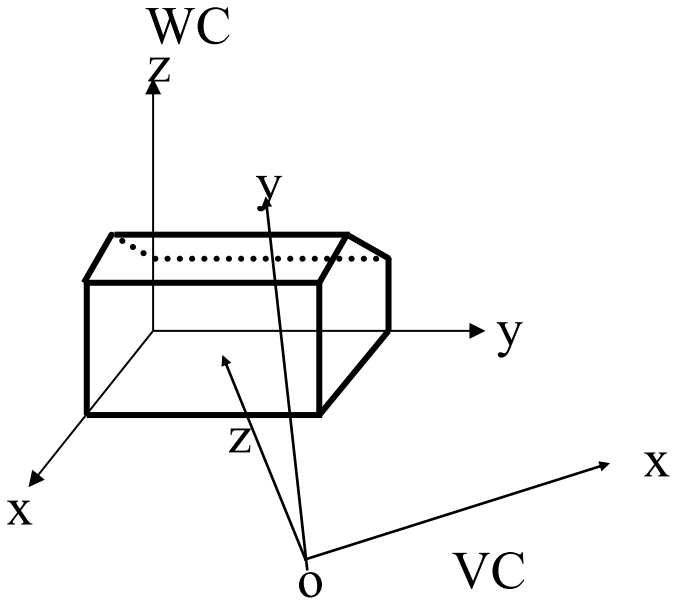
- **Model-View Transformation**
  - –Model Transformation模型变换*
  - –**View Transformation观察变换***
- Projection Transformation
- Viewport Transformation

The University of New Mexico

View Transformation*

- **View Transformation**
  - 从世界坐标系到观察坐标的变换 (from WC to VC)
  - “观察变换”:也称“视点变换”,“相机变换”,“眼坐标变换”





The University of New Mexico

View Transformation（cont.）

➢坐标系变换实质是框架变换（Change of Frames）



- Extending what we did with change of bases

$$u_1 = \gamma_{11}v_1+\gamma_{12}v_2+\gamma_{13}v_3$$
$$u_2 = \gamma_{21}v_1+\gamma_{22}v_2+\gamma_{23}v_3$$
$$u_3 = \gamma_{31}v_1+\gamma_{32}v_2+\gamma_{33}v_3$$
$$Q_0 = \gamma_{41}v_1+\gamma_{42}v_2+\gamma_{43}v_3+\gamma_{44}P_0$$

- Within the two frames, any point or vector has a representation of the same form

$\mathbf{a}=[\alpha_1\ \alpha_2\ \alpha_3\ \alpha_4]$ in the first frame(u1,u2,u3,Q0)
$\mathbf{b}=[\beta_1\ \beta_2\ \beta_3\ \beta_4]$ in the second frame(v1,v2,v3,P0)
where $\alpha_4=\beta_4=1$ for points and $\alpha_4=\beta_4=0$ for vectors .
defining a 4 x 4 matrix

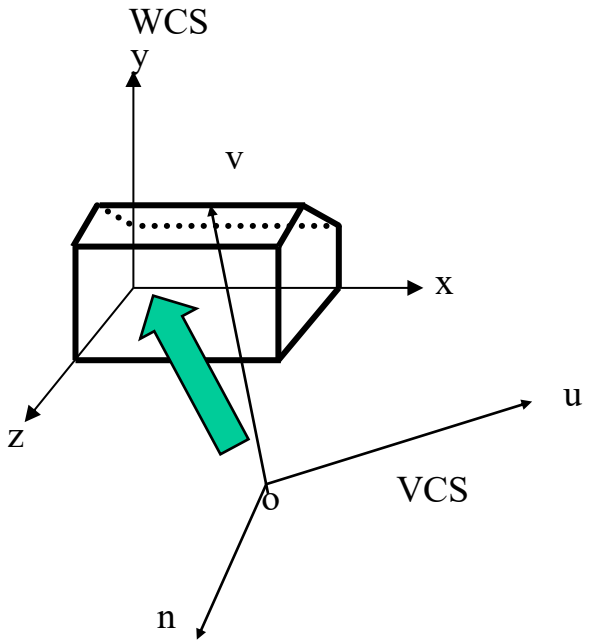
$$\mathbf{a}=\mathbf{bM} \qquad \mathbf{M}=\begin{bmatrix}\gamma_{11} & \gamma_{12} & \gamma_{13} & 0\\ \gamma_{21} & \gamma_{22} & \gamma_{23} & 0\\ \gamma_{31} & \gamma_{32} & \gamma_{33} & 0\\ \gamma_{41} & \gamma_{42} & \gamma_{43} & 1\end{bmatrix}$$

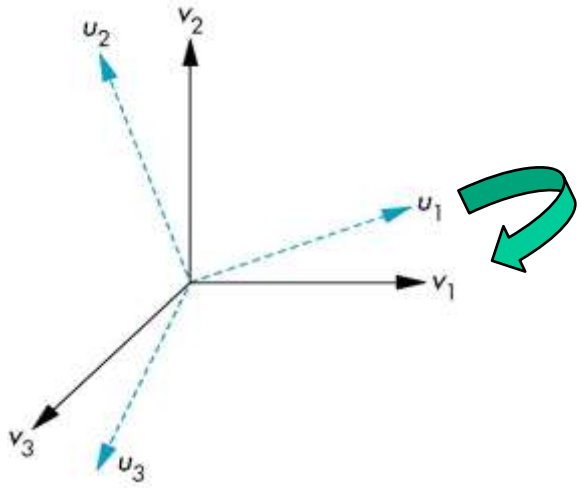
- Here a , b are row vector representation
- The matrix **M** is 4 x 4 and specifies an affine transformation in homogeneous coordinates

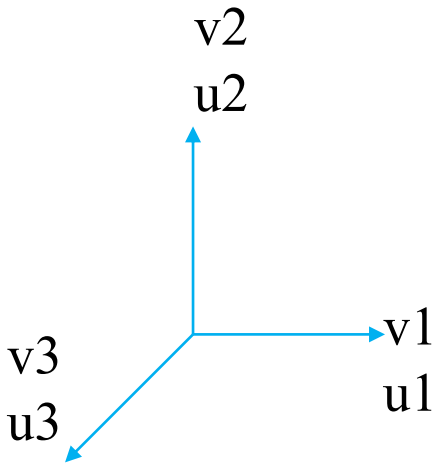
The University of New Mexico

View Transformation（cont.）

- 观察变换（串联变换矩阵）的推导方法
- View Transformation is the Combination transformation that makes the VCS and WCS coincide(重合): **"move VCS to WCS"， Commonly like: M= R * T**



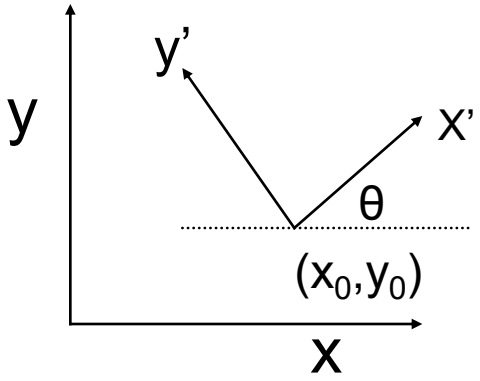




View Transformation（cont.）

➢2D View Transformation

- How to transform the representations of object's locations from XY to X'Y'?
- Example:



✓思路：将 X'O'Y' 变换到与XOY重合

1.平移 $(-x_0,-y_0)$到(0,0)：$T(-x_0,-y_0)$

2.旋转x'y'到xy：$R(-\theta)$

此观察变换矩阵为：　$M=R(-\theta)T(-x_0,-y_0)$

View Transformation（cont.）

➢2D View Transformation(cont.)

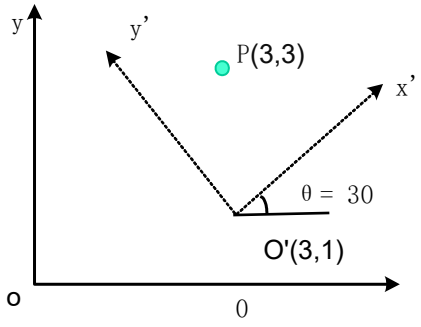
➢Example：O’(3,1),右手坐标系统，Pwc=(3,3),求Mview=?， Pvc =?

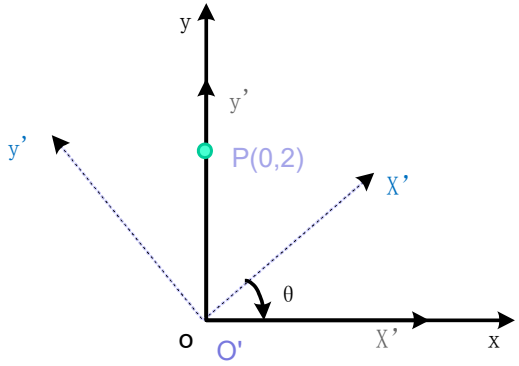
1：将x‘y’坐标系的原点平移到xoy坐标系的原点：T(-3,-1)

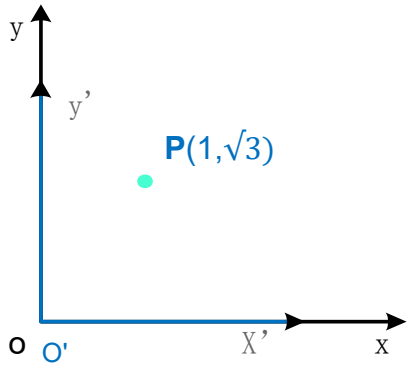
2: 顺时针旋转30度：R(-30)

M_{view}= R(-30) *T(-3,-1)

P_{vc} = R(-30) *T(-3,-1)* P_{wc} =（1，$\sqrt{3}$）







View Transformation（cont.）

2D View Transformation（cont.）

？ Mview=R*T，不知道旋转角，R如何求？

- 设R中的第一行为一个向量u，第二行为v，
- 发现u,v正好是VC坐标系的方向轴上的单位正交向量，因：u·v=0，|v|=1，|u|=1 ；
- 所以，旋转变换矩阵R可以直接写为VC坐标系的两个方向轴上的单位向量！

$$R=\begin{bmatrix}\cos\theta & -\sin\theta \\ \sin\theta & \cos\theta\end{bmatrix} \Rightarrow R=\begin{pmatrix}u_x, u_y \\ v_x, v_y\end{pmatrix}$$

View Transformation（cont.）

➢2D View Transformation(cont.)

Example：已知O’(3,1),右手坐标系统，Pwc=(3,3),求Mview=?， Pvc =?

➢计算O’X’上的单位向量u=$(\frac{\sqrt{3}}{2}, 1/2)$,

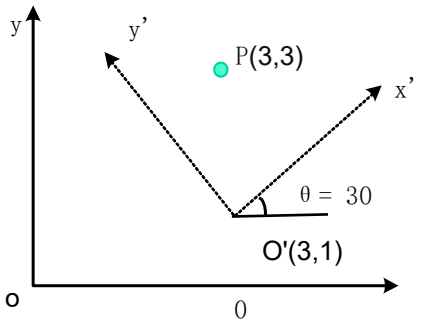
➢计算O’Y’上的单位向量v=(-1/2, $\frac{\sqrt{3}}{2}$)

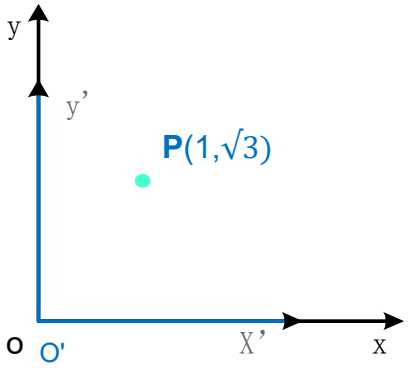
绕原点逆时针转90度得到单位向量v=(v_x,v_y)= (-u_x, u_y)

R(u,v)= $\begin{bmatrix} u_x, u_y \\ v_x, v_y \end{bmatrix}$

➢Mview= R(u,v) *T(-3,-1)

➢Pvc = R(u,v) *T(-3,-1)* Pwc =（1， $\sqrt{3}$ ）





View Transformation（cont.）

➢2D View Transformation(cont.)

Practice:O’在XOY坐标系中的坐标为（0,0），点V是O’ Y’轴上的一点，V(3,4)。请根据单位向量构造旋转矩阵的方法，求从xoy变换到x’o’y’的观察变换矩阵。（新旧坐标都是右手系坐标系统）

➢计算O’Y’单位向量v:

➢ $v=(v_x,v_y)=V/|V| =(3-0,4-0) / 5 =（3/5，4/5）$ //$|V|=5$

➢计算O’X’上的单位向量u：

➢v绕原点顺时针转90度得到单位向量u: $u=(u_x,u_y)=v*R(-90)= (v_y,-v_x)$

➢3）将u，v的分量带入R矩阵

$$R(u,v)=\begin{pmatrix} u_x,u_y,0 \\ v_x,v_y,0 \\ 0,\ 0,1 \end{pmatrix}$$

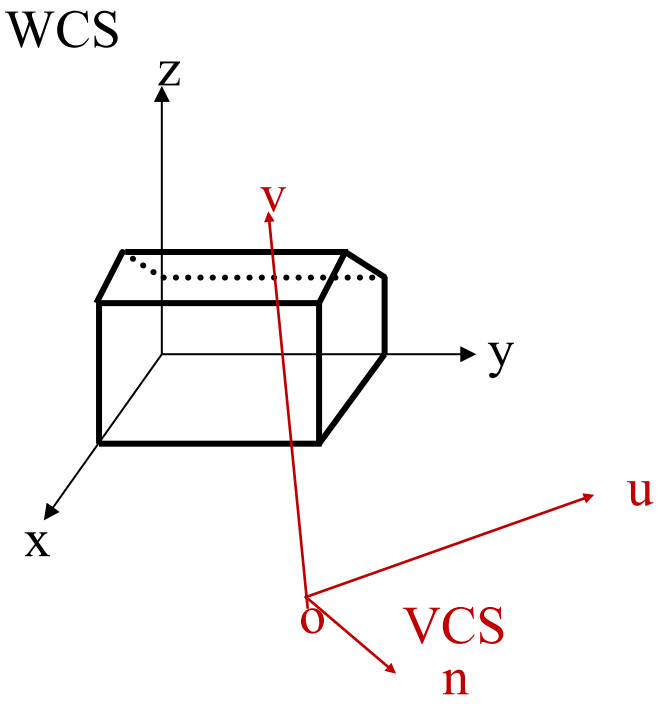
$$R=\begin{pmatrix} 4/5, & -3/5, & 0 \\ 3/5, & 4/5, & 0 \\ 0\ , & 0\ , & 1 \end{pmatrix}$$

The University of New Mexico

View Transformation（cont.）

- 3D View Transformation
  - 根据推导: $M_{wc->vc} = R \cdot T = R \cdot T(-eye)$,
  - R=?　　//R(u,v,n)

$$R = \begin{bmatrix} u_1 & u_2 & u_3 & 0 \\ v_1 & v_2 & v_3 & 0 \\ n_1 & n_2 & n_3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



View Transformation（cont.）

The University of New Mexico

- 3D View Transformation(cont.)
  - Find R in 3D: Finding u, v, n from Eye, At and Up

n: n和观察方向Look相反（VC是右手坐标系）

$n=N/|N|=(n_1,n_2,n_3)$ //N= - Look = - (At-Eye)

$$n=\frac{-Look}{\|Look\|}$$

u: 垂直于Up向量和n向量所在的平面

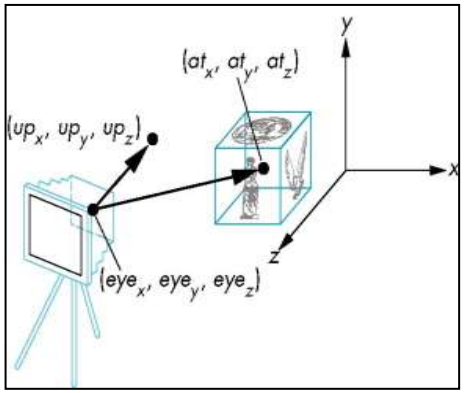
$u=(Up \times N)/|Up \times N|=(u_1,u_2,u_3)$

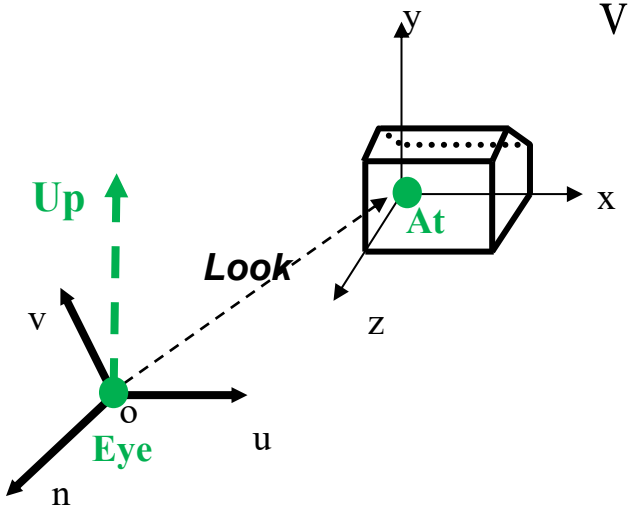
$$u=\frac{Up\times n}{\|Up\times n\|}$$

v: 通过n和u的叉乘得到

$v=n \times u=(v_1,v_2,v_3)$

$$v=n\times u$$

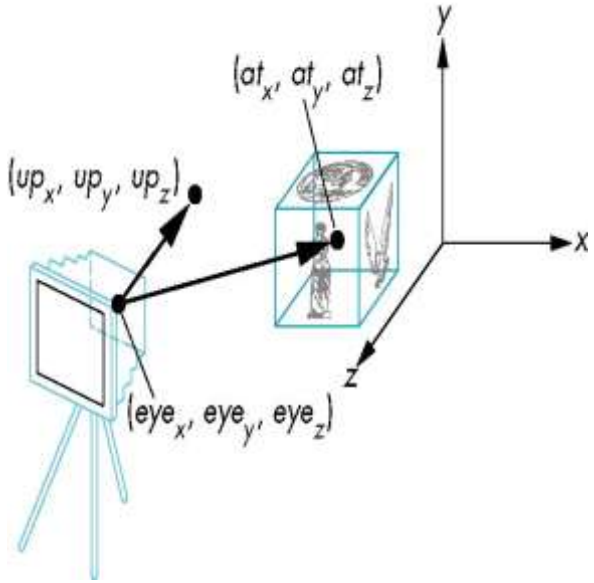






View Transformation (cont.)

- 3D View Transformation(cont.)
  - **Find R in 3D: Finding u, v, n from Eye, At and Up**
  - In fact, $M_{wc->vc} = R \cdot T(-eye) =$ **lookAt(Eye, At , Up)**




```
//LookAt(eye, at, up)

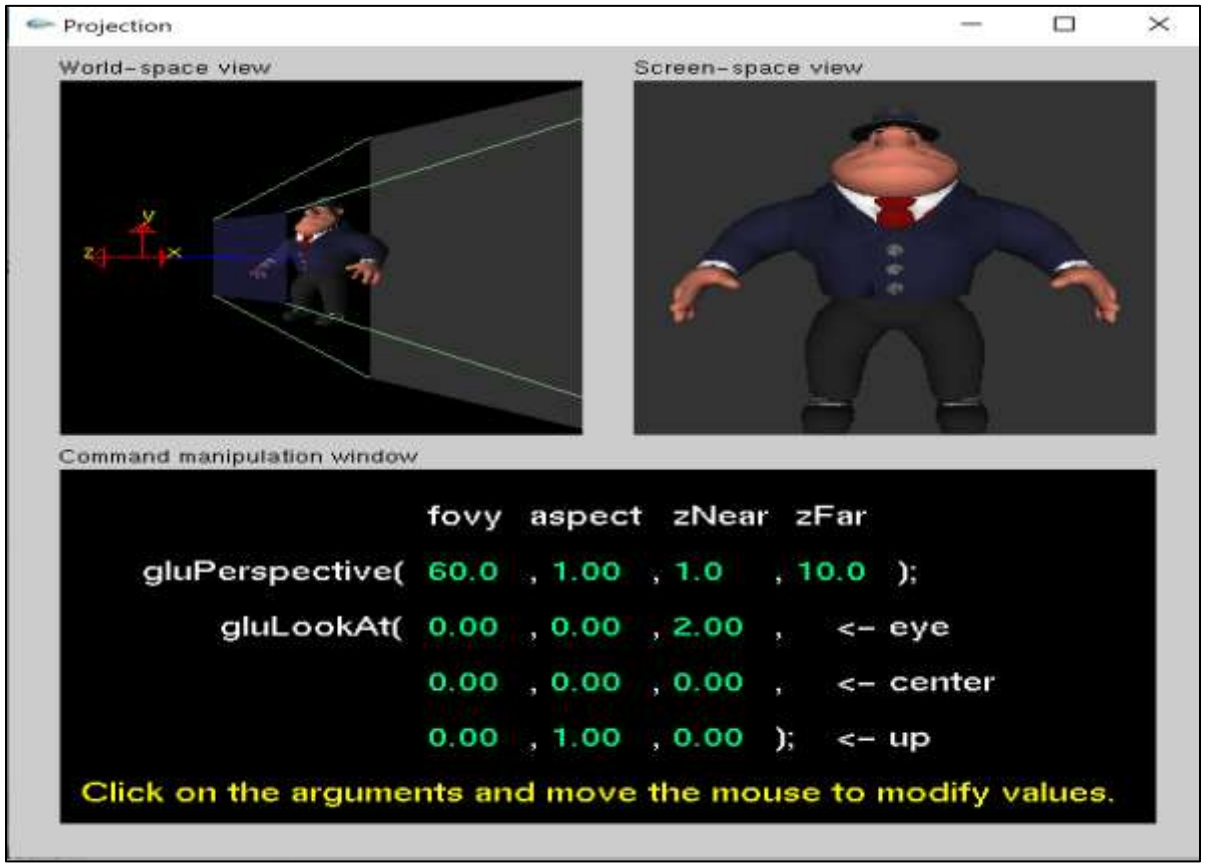
var eye = vec3(1.0, 1.0, 1.0);
var at = vec3(0.0, 0.0, 0.0);
var up = vec3(0.0, 1.0, 0.0);

var mv = LookAt(eye, at, up);
```



View Transformation（cont.）

- 3D View Transformation(cont.)
  - gluLookAt (Eye,At,Up) Function (OpenGL)
    - The GLU library contained the function **gluLookAt**() to form the required modelview matrix through a simple interface API function .





View Transformation（cont.）

➢3D View Transformation(cont.)

- LookAt (Eye,At,Up) In AngelCode/common/MV.js (webGL)

```
function lookAt( eye, at, up )
{
    if ( !Array.isArray(eye) || eye.length != 3) {
        throw "lookAt(): first parameter [eye] must be an a vec3";
    }

    if ( !Array.isArray(at) || at.length != 3) {
        throw "lookAt(): first parameter [at] must be an a vec3";
    }

    if ( !Array.isArray(up) || up.length != 3) {
        throw "lookAt(): first parameter [up] must be an a vec3";
    }

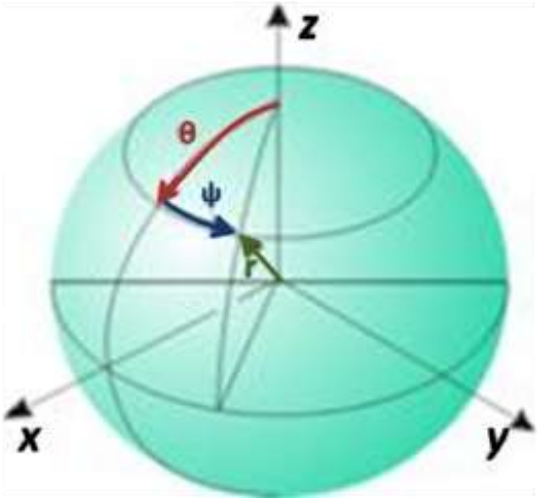
    if ( equal(eye, at) ) {
        return mat4();
    }
    //下面的单位向量名字调整为(u,v,n)，原来ANGEL是(n,u,v),并且进行了调整
    var n = negate(normalize( subtract(at, eye) ));  // view direction vector
    var u = normalize( cross(up, n) );               // perpendicular vector
    var v = normalize( cross(n, u) );                // "new" up vector

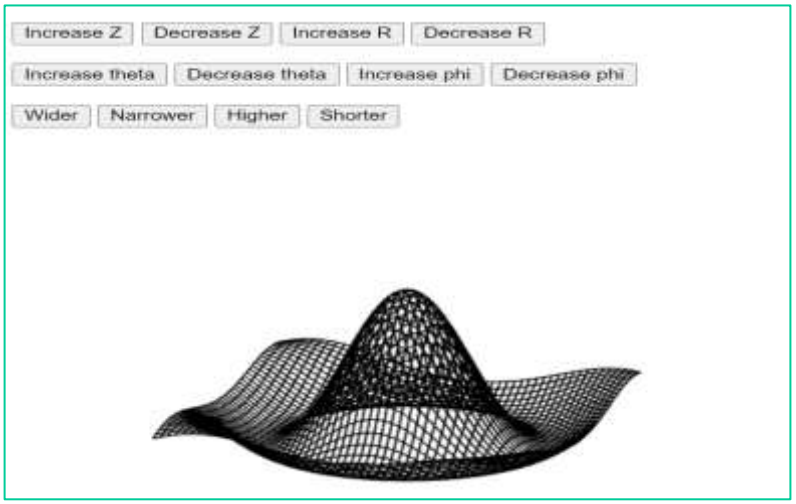
    //result=R(u,v,n)*T(-eye)
    var result = mat4(
        vec4( u, -dot(u, eye) ),
        vec4( v, -dot(v, eye) ),
        vec4( n, -dot(n, eye) ),
        vec4()
    );
    return result;
}
```

View Transformation（cont.）

- 观察变化的例子：相机在球面移动，漫游观察物体
  ref: //AngelCode/Chap5/hata.html
  - 相机位置eye(x,y,z)的定位
    X=r*sin(theta)cos(phi)
    Y=r *sin(theta)sin(phi)
    Z=r *cos(theta)









View Transformation（cont.）

```
window.onload = function init() {
        ……
    ViewMatrixLoc = gl.getUniformLocation(program, "ViewMatrix");
    ……}
```

```
var render = function() {
      ViewMatrix = lookAt(eye, at , up);
      gl.uniformMatrix4fv(ViewMatrixLoc, false, flatten(ViewMatrix) );
      …….}
```

```
//vertex  shader
Uniform Mat4 ViewMatrix;
void main() {
      //输出的顶点坐标gl_Position，应该是裁剪坐标系下的坐标！
      gl_Position =projectionMatrix*ViewMatrix* ModelMatrix vPosition;
      ……
}
```

Summary

➢类比于相机拍摄过程，经历“五”种坐标系和“四”种变换！

物体模型

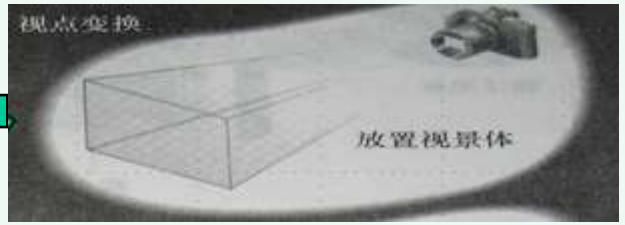
■MC模型坐标

将物体建模坐标描述转换到世界坐标场景下描述-模型变换



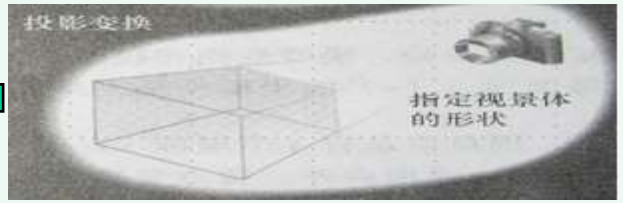
■WC世界坐标

将世界坐标场景描述转换为观察坐标系描述-观察变换/相机变换/眼坐标变换



■VC观察坐标

将观察坐标裁剪窗口内景物进行规范化投影变换-规范化投影变换+裁剪



■NDC规范化坐标

将规范化区域景物映射到设备坐标系下视区内-屏幕映射/视口变换



■DC设备坐标

■输出设备

Summary（cont.)

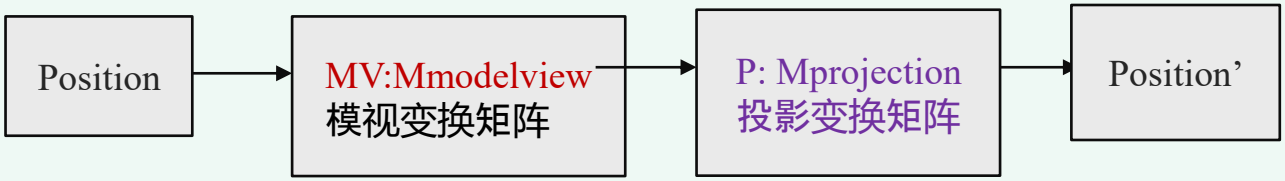
- Model-View Transformation

- Simple：Instance Transformation 简单的实例变换
  - **General：M=TRS**
- Complex Model Transformation 复杂的模型变换
  - **层次建模：铰链模型，太阳系等**

- 观察变换 (相机变换/眼坐标变换/视点变换）
  - **在WC中建立VC，推导观察变换矩阵**
    - V= R * T=R（u，v，n) * T(-eye)
    - 常用CG API函数：LookAt (eye, at, up);

Summary（cont.）

- **顶点着色器中的变换：**
  - Position'= PVM * Position;



  - PVM = Projection * View * Model=Project * ModelView

- 模视变换矩阵 ModelView:
  - 将几何对象的表示从建模坐标MC变换到相机坐标VC
    - **Model Transformation: MC->WC,**
    - **View Transformation : WC->VC**



View Transformation（cont.）

- **GLSL编程注意事项：**
  - **“行先序”：ans[3][0]表示是第4行第1列**
  - **“列先序”：ans[3][0]表示是第4列第1行**

- 主语言数组矩阵表示通常是“行”先序表示
- 但GLSL中的矩阵是“列”先序表示！
  - GLSL中的矩阵存储是按列先序（column-major）存储的，这意味着矩阵的数据是按照“列顺序”连续存储在内存中的（openGL相同）

例如：glm::mat4 ans = glm::mat4(1.0f);

ans[3][0] = xTranslationValue;

若按“列先序”：ans[3][0]表示是第4列第1行

- 所以：一般在GLSL中进行矩阵运算时，通常需要将行主序（row-major）存储的矩阵转换为列主序(column-major)，以确保正确的矩阵运算结果。（可进行适当的“转置”操作）